WHY SEPARATE POLICIES FOR EMPLOYEES AND CUSTOMERS?

It might seem easier to combine employee and customer access into one policy. For example, you could create a single permit that covers both groups and then use a branch inside the `when` clause to determine which rules apply. However, Cedar doesn't support unions in the policy scope—you can't specify `principal in [ACME::Employee, ACME::Customer]`. To include both, you need to leave the principal unrestricted and push the type discrimination into the `when` clause.

That works, but it makes the policy harder to read and maintain because the branching logic lives in one big condition. By contrast, splitting into two policies lets you keep tight scopes, like `principal is ACME::Employee` for the employee rule and `principal is ACME::Customer` for the customer rule, which is clearer and easier to test. This also allows Cedar to more easily use the scope to determine which policies to include in the evaluation.

Here's what a combined policy would look like (not recommended):

```
// Combined employee+customer view (illustrative)
permit(
  principal,                              // no union in scope
  action == ACME::Action::"doc:view",
  resource is ACME::Document
)
when {
  // Employees: readers team or manager of owner
  (principal is ACME::Employee &&
   (principal in resource.employee_readers_team ||
    resource.owner.manager == principal))
  ||
  // Customers: readers team
  (principal is ACME::Customer &&
   principal in resource.customer_readers_team)
};
```

Splitting the policies avoids this branching and keeps each policy focused on a single principal type, which improves readability and reduces the